

Звіт з лабораторної роботи №4

Тема: Статичний аналіз коду

Студент: Мамедов Арзу

1. Знайдені проблеми в коді:

№	Де проблема	В чому суть	Як виправити
1	pay.py:2	Не вказані типи змінних у методах <code>order_id, items, discount_percent</code>	Додати типи даних, щоб було зрозуміліше, що передавати в функції
2	pay.py:12	Перевірка знижки знаходиться всередині обчислення суми, через що метод виглядає громіздким	Винести перевірку знижки в окремий приватний метод <code>_validate_discount</code>
3	pay.py:18	Для підрахунку суми використовуємо <code>for</code>	Замінити цикл на функцію <code>sum()</code>
4	pay.py:26	Словник з промокодами написаний прямо всередині функції	Винести промокоди в константу класу <code>PROMO_CODES</code>

2. Що було змінено в коді:

Додав типи змінних:

Було - `def __init__(self, order_id, items, discount_percent=0.0):`

Стало - `def __init__(self, order_id: int, items: list, discount_percent: float = 0.0):`

Це було зроблено для того щоб редактор коду підказував правильні типи даних

Спростив підрахунок суми:

Було:

Python

```
subtotal = 0
```

```
for item in self.items:
```

```
    subtotal += item["price"] * item["qty"]
```

Змінено на

```
subtotal = sum(item["price"] * item["qty"] for item in self.items)
```

Виніс промокоди в константу:

Словник створював кожного разу всередині функції **apply_promo_code**

Та ми створили змінну **PROMO_CODES** на рівні класу

Це вже було зроблено щоб не дублювати дані при кожному виклику функції

3. Перевірка тестами:

```
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\windo\OneDrive\Desktop\лаба3
plugins: cov-7.1.0
collected 10 items

test.py ..... [100%]

===== 10 passed in 0.06s =====
PS C:\Users\windo\OneDrive\Desktop\лаба3>
```

4. Порівняння метрик коду

Показник	До виправлень	Після виправлень
Складність коду	В були складні функції	А усі функції стали простими
Помилки линтера	Знайдено помилки	All checks passed!

Скриншоти перевірок:

1. Складність коду до виправлень:

```
PS C:\Users\windo\OneDrive\Документи\GitHub\lab1> cd "C:\Users\windo\OneDrive\Desktop\лаба3"
PS C:\Users\windo\OneDrive\Desktop\лаба3> python -m radon cc pay.py -s
pay.py
M 11:4 Order.calculate_total - A (5)
C 1:0 Order - A (4)
M 33:4 Order.pay - A (3)
M 2:4 Order.__init__ - A (2)
M 25:4 Order.apply_promo_code - A (2)
PS C:\Users\windo\OneDrive\Desktop\лаба3>
```

2. Складність коду після виправлень:

```
PS C:\Users\windo\OneDrive\Desktop\лаба3> python -m radon cc pay.py -s
pay.py
C 1:0 Order - A (3)
M 18:4 Order.calculate_total - A (3)
M 34:4 Order.pay - A (3)
M 4:4 Order.__init__ - A (2)
M 13:4 Order._validate_discount - A (2)
M 28:4 Order.apply_promo_code - A (2)
PS C:\Users\windo\OneDrive\Desktop\лаба3>
```

3. Перевірка линтером:

```
PS C:\Users\windo\OneDrive\Desktop\лаба3> python -m ruff check pay.py
All checks passed!
PS C:\Users\windo\OneDrive\Desktop\лаба3>
```

5. Висновок

В ході лабораторної роботи я навчився шукати слабкі місця в коді за допомогою програм **ruff** та **radon**